

Team Task Distribution

SIH Internal Round Prototype | PS 34 — Legal Metrology Automation

Team size: 6 members — 3 on Main Tasks (core, interdependent systems), 3 on Small Tasks (essential but lower-complexity builds).

Why 3+3 instead of 6 separate tracks: Only three pieces of this system are genuinely complex and tightly interdependent — the backend, the AI/OCR/NLP engine, and the mobile app. Keeping those three owned by three people means there are only three interfaces to keep in sync (API contracts, OCR response format, notice template format). The other three members build against those contracts (or mock data, early on) rather than adding three more interfaces to coordinate — that's what keeps integration simple instead of a 6-way coordination problem.

MAIN TASKS (3 members)

1. Backend & Database Architect

Owns: NestJS API Gateway, PostgreSQL schema, workflow/state logic

- Design the core database schema: users, businesses, complaints, cases, inspections, notices, offence records
- Build REST APIs for all four dashboards (complaint creation, inspection creation, notice status updates, dashboard stats)
- Set up Keycloak roles and permissions (Citizen / Business / Inspector / Controller)
- Build the case workflow/state machine: Received → Assigned → Inspected → Notice Issued → Compounded/Prosecution → Resolved
- Build the first/second-offence lookup logic (queries against product + manufacturer history)
- Build the statutory deadline calculation (days remaining per notice type)
- Build the compounding order Approve/Reject workflow for the Controller

Depends on:

Rulebook structure from the Database/Knowledge Base Maker (penalty and offence-tier data)

Demo deliverable:

Working APIs — file a complaint, create an inspection, issue a notice, check offence history, approve/reject a compounding order.

2. AI/OCR + NLP Engineer

Owns: Python + FastAPI AI service

- Image preprocessing: merge/handle multiple angle photos of one product's packaging
- OCR text extraction (Tesseract / EasyOCR / a cloud Vision API — whichever is fastest to get reliable for the demo)
- Information extraction: pull MRP, net quantity, mfg/exp date, manufacturer address, etc. out of raw OCR text (regex + light NLP)
- Compliance rule engine: compare extracted fields against the Rulebook to flag violations
- Fuzzy text matching (e.g. rapidfuzz) on product name + manufacturer name, to power the first/second-offence check without needing barcode infrastructure
- NLP-based Notice Generator: fills the four notice templates (Improvement, Seizure, Panchanama, Compounding) using case data + the legal knowledge base

Depends on:

Rulebook data and notice templates from the Database/Knowledge Base Maker — this is the single most important dependency in the whole project, so sync with them from day one.

Demo deliverable:

An endpoint that takes 2–3 packaging photos and returns extracted fields + a violation report; a notice generator endpoint that outputs a filled notice from case data.

3. Mobile App Developer (Flutter — Inspector + Business)

Owns: The Flutter app, both Inspector and Business modes

- **Inspector mode:** business search/select screen; multi-angle camera capture for packaging scan; editable OCR-results review screen (inspector confirms/corrects before generating a notice); Panchanama form (two witness fields, seizure sample photo + auto-generated Sample ID); notice generation trigger + digital signature step; offline mode with local storage and sync-on-reconnect
- **Business mode:** registration screen; packaging self-check (camera capture → private results screen with recommendations); notice inbox — view notices, submit dispute, resubmit corrected photos, give consent, pay via Razorpay

Depends on:

APIs from Member 1, OCR/NLP endpoints from Member 2

Demo deliverable:

A single Flutter app demoing both the full inspection flow and the full business self-check flow, end to end.

SMALL TASKS (3 members)

4. Web Frontend Developer (Next.js — Citizen + Controller)

Owns: Web dashboards for Citizen and Controller

- **Citizen side:** OTP signup; complaint filing form (category dropdown, photo/invoice upload, retailer name/address with autocomplete); status tracker (stepper UI)
- **Controller side:** statewide dashboard (first vs. second offence counts, region-wise case load); compounding order review screen (Approve / Reject + comment, or Escalate to Prosecution); supply-chain inspection assignment screen

Depends on:

APIs from Member 1

Demo deliverable:

A working web app to file a complaint as a citizen, and to approve/reject a case as the Controller.

5. Database / Legal Knowledge Base Maker

Owns: The Legal Metrology rulebook — the most important supporting role

- Read the Legal Metrology Act, 2011 and the Legal Metrology (Packaged Commodities) Rules, 2011
- Build a structured rulebook (JSON or a spreadsheet the team can import): mandatory label field, exact requirement, applicable product category, Act/Rule section number, and penalty for violation
- Build the first-offence vs. second-offence penalty table
- Draft the 4 notice templates (Improvement, Seizure, Panchanama, Compounding) with legally accurate section references and placeholder fields for the NLP engine, in English and one regional language

Depends on:

Works closely with Member 2 (needs this data to build the rule engine) and Member 1 (needs offence/penalty data for the workflow logic)

Demo deliverable:

A rulebook.json file and 4 notice template documents, ready to plug into the system.

Note: this is not "small" in importance — it's smaller in coding effort, but the entire OCR compliance engine and notice generator are useless without it. This role is research-and-structuring heavy rather than pure development,

so it suits a teammate strong at reading regulation carefully and organizing it cleanly — but the output must be structured data, not a summary PDF.

6. Integrations, QA & Presentation Owner

Owns: Payments, mocked integrations, testing, and the pitch

- Razorpay integration (sandbox/test mode) for business penalty payments
- Mock/stub the remaining external integrations for the demo — WhatsApp/Email notification triggers, GSTIN validation (a simple pass/fail check is enough), eMudhra digital signature (a basic "sign" action storing a signature image is enough for an internal round)
- End-to-end testing across all four dashboards before demo day — clicking through the full citizen-to-controller flow repeatedly to catch the bugs where the three main-task members' pieces don't quite line up
- Prepares the SIH Internal Round presentation deck and rehearses the live demo script

Demo deliverable:

A working payment flow, a clean bug-free run-through of the entire flow, and the presentation deck.

SUGGESTED BUILD ORDER

Phase	Focus
Week 1	Member 5 starts the Rulebook immediately — everyone else needs it. In parallel: Member 1 finalizes the DB schema, Member 2 sets up the OCR pipeline skeleton, Member 3 scaffolds the Flutter app, Member 4 scaffolds the Next.js app, Member 6 sets up the Razorpay sandbox.
Week 2	Member 1 builds out the real APIs. Member 2 builds the rule engine and notice generator using the Rulebook. Members 3 and 4 build UI screens against mock data.
Week 3	Integration — connect the Flutter and Next.js apps to real APIs and the OCR/NLP service. Wire up the full notice ladder, dispute flow, and correction resubmission.
Week 4	Member 6 leads full end-to-end testing, the team fixes integration bugs together, and the presentation is finalized.

Adjust the week count to whatever your actual internal-round deadline gives you — the order (Rulebook first, then core APIs/OCR, then UI, then integration/testing) matters more than the exact timing.